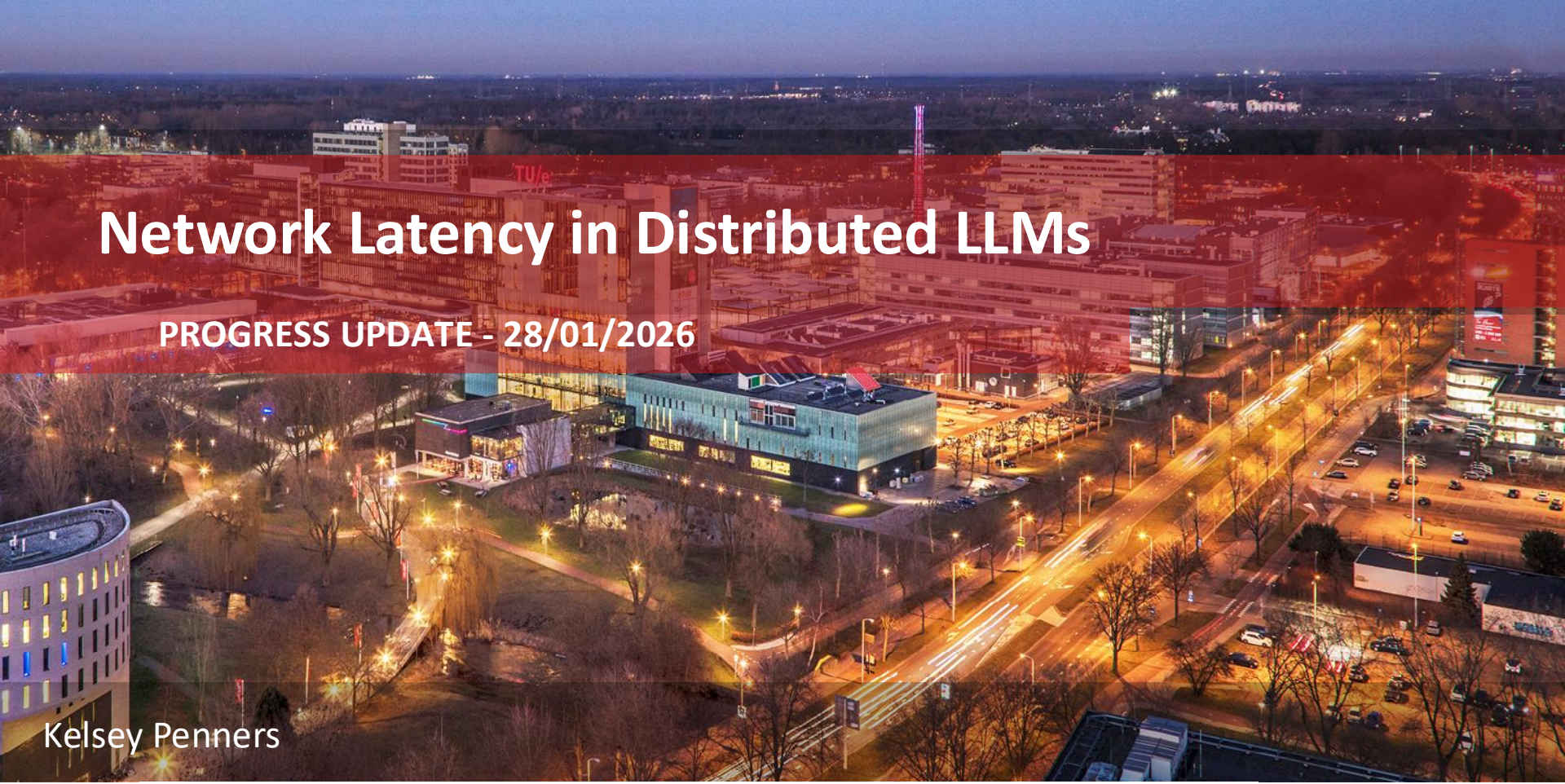


Network Latency in Distributed LLMs

PROGRESS UPDATE - 28/01/2026

Kelsey Penners

Project Overview

Characterize how communication latency affects distributed LLM performance

- Phase 1: Single GPU
- Phase 2: Multi-GPU within server
- Phase 3: Multi-GPU, Multi-server

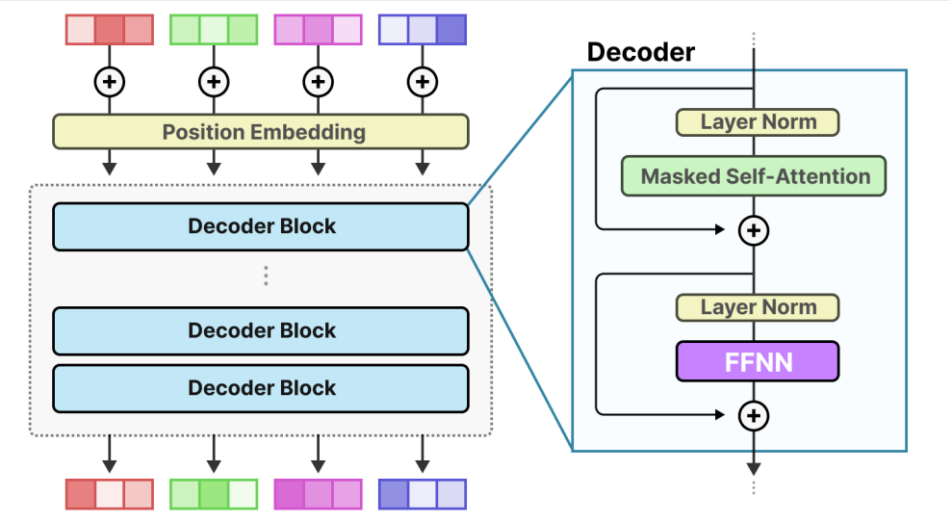
Where I'm at:

- Learning foundational concepts needed to understand how communication works within distributed LLMs

LLM Basics

Two Main Phases

- Prefill: all tokenized input processed simultaneously, generate and store in the KV cache
- Decode / Generation: output tokens get generated, reuse KV cache

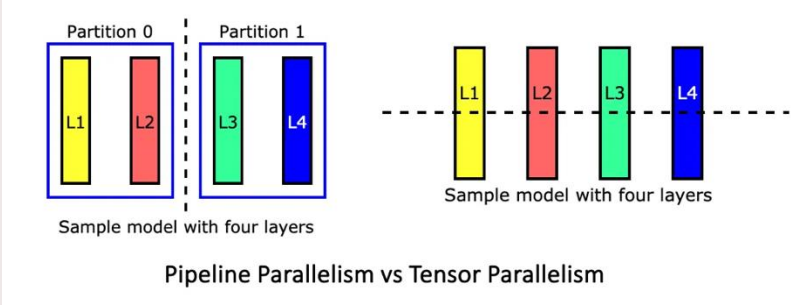


https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts

Parallelism – how LLMs can be distributed

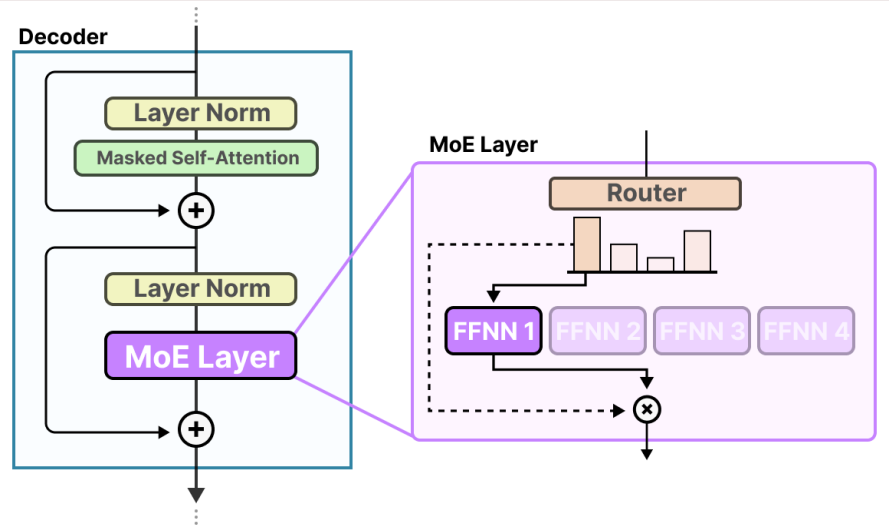
- *Data parallelism*: each GPU has a copy of model and slice of data
- *Tensor parallelism*: model is cut horizontally
  - Communication done using **allreduce**
  - Useful in multi-GPU set ups
- *Pipeline parallelism*: model is cut vertically, whole layers on GPUs
  - Useful in multi-GPU, multi-server set ups
  - Point-to-point transfers between stages

Allreduce: all GPUs aggregate their data and receive result



https://medium.com/tr-labs-ml-engineering-blog/tensor-parallel-llm-inferencing-09138daf0ba7

Mixture of Experts (MoE) Basics



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts

- *Standard LLMs:* attention block followed by FFNN (MLP) repeated
- *MoEs:* attention block, router, and experts
- Network latency could affect these models differently
  - Potential for multi model analysis?

Metrics

- TTFT (time to first token): measures prefill phase efficiency
- Inter-token latency: decode phase
- Throughput (tokens/sec): overall performance

Possible plot analysis:

- TTFT vs ISL (input sequence length): how does prefill perform?
- Inter-token latency vs token position: is decode stable?
- Performance degradation vs latency injection

Goal: to isolate where communication latency impacts performance most

Frameworks & Tools

Tool	Purpose
vLLM	LLM inference serving w/ PagedAttention
tensorRT-LLM	Inference engine, NVIDIA optimized
tc - netem	Network latency injection for experiments
DeepSpeed-Inference	Distributed inference w/ tensor/pipeline parallelism support
NCCL	GPU communication library

Next steps

- More about KV cache: how often it gets accessed, synchronization across GPUs
- Paged Attention, Flash Attention: other transformer optimizations
- Megatron paper for parallelism implementation
- Narrow down what frameworks and models work for this project
- Understand network topology and hardware characteristics
- Explore and experiment with tools

Key Literature (so far)

Title	Topic	Relevancy?
Relevance of Latency in Ethernet Networking for AI Infrastructure	Studies impact of network latency in training, showing little performance degradation.	Mirrors this project, but focuses on LLM training. Network simulation
DeepSpeed Inference	Explores tensor parallelism within nodes, pipeline parallelism across nodes to optimize inference.	Distributed architecture patterns
Efficient Memory Management for LLM Serving with Paged Attention	Paged Attention for KV cache, vLLM.	KV cache flow is critical for multi-GPU LLMs
SimAI	Network simulator for LLM training, models NCCL primitives.	Inspiration for controlled latency experimentation

Research gap in how inference is affected by network latency.
Relevant papers in latency impact of training and inference optimization.

References & Resources

Key papers:

- Relevance of Latency in Ethernet Networking for AI Infrastructure - *https://ieeexplore.ieee.org/abstract/document/10926143*
- DeepSpeed Inference - *https://arxiv.org/abs/2207.00032*
- Efficient Memory Management for LLM Serving with Paged Attention - *https://arxiv.org/pdf/2309.06180*
- SimAI - *https://www.usenix.org/system/files/nsdi25-wang-xizheng-simai.pdf*
- Megatron - *https://arxiv.org/pdf/1909.08053*

Thank you